

Bidly

Dossier de remise - Projet React Native

Application mobile d'enchères simples

Étudiant : Thomas Paquet

1. Rappel du sujet

Bidly est une application mobile d'enchères simples développée avec React Native. Elle permet à un utilisateur de consulter des objets mis aux enchères, de rechercher un objet par mot-clé, de consulter le détail d'une enchère, de placer une offre, de suivre les enchères auxquelles il participe et d'être notifié lorsqu'un autre utilisateur surenchérit.

L'application est pensée comme un prototype scolaire réaliste. Elle montre le fonctionnement principal d'un système d'enchères, mais elle ne cherche pas à devenir une marketplace complète. Les paiements, la livraison, les litiges, les évaluations de vendeurs et la messagerie ne sont pas gérés.

Le projet met l'accent sur un parcours clair : un utilisateur peut créer un compte, consulter les enchères disponibles, créer sa propre vente, enchérir sur une vente d'un autre utilisateur, suivre ses offres et recevoir une notification interne lorsqu'il est dépassé.

2. Objectifs fonctionnels

Le premier objectif est de proposer une liste d'enchères lisible. Chaque enchère doit afficher les informations utiles pour décider si l'utilisateur veut ouvrir le détail : titre, image si disponible, prix actuel, vendeur et statut.

Le deuxième objectif est de permettre la recherche par mot-clé. La recherche reste simple et filtre les enchères visibles selon le texte saisi par l'utilisateur.

Le troisième objectif est de permettre le placement d'une offre. Le montant doit être un nombre valide, supérieur au prix actuel, et l'enchère doit encore être active. Un utilisateur ne peut pas enchérir sur sa propre vente.

Le quatrième objectif est de suivre les enchères auxquelles l'utilisateur a participé. L'écran Mes offres permet de voir son dernier montant, le prix actuel et si l'utilisateur est encore en tête.

Le cinquième objectif est de notifier un utilisateur lorsqu'un autre compte place une offre plus élevée que la sienne. La notification est interne à l'application afin de rester simple et fiable pour le projet.

3. Fonctionnalités implémentées

Les fonctionnalités présentes dans la version remise sont les suivantes.

- Affichage d'une liste d'enchères actives.
- Recherche par mot-clé sur les enchères affichées.
- Écran de détail pour consulter les informations d'une enchère.
- Inscription et connexion avec email et mot de passe via Supabase Auth.
- Affichage du nom de l'utilisateur connecté plutôt que son adresse email.
- Création d'une enchère par un utilisateur connecté.
- Ajout d'une image à une enchère via une URL ou le choix d'une image depuis l'appareil.
- Suppression d'une vente par son vendeur.
- Placement d'une offre sur une enchère active.
- Validation des offres trop basses, invalides ou placées après la fin de l'enchère.
- Blocage des offres sur sa propre vente.
- Historique des dernières offres sur le détail d'une enchère.
- Écran Mes offres pour suivre les enchères auxquelles l'utilisateur participe.
- Notifications internes lorsqu'un autre utilisateur surenchérit.
- Badge visuel indiquant le nombre de notifications non lues.
- Marquage des notifications comme lues.
- États de chargement, d'erreur et de liste vide.

La version finale utilise Supabase comme source de données unique. Il n'y a pas de jeu de données local de secours : si Supabase n'est pas configuré ou indisponible, l'application affiche une erreur plutôt que de montrer de fausses données.

4. Authentification et profil

L'authentification est gérée par Supabase Auth. L'utilisateur peut créer un compte avec un nom, une adresse email et un mot de passe. Après la connexion, l'application conserve l'utilisateur courant dans un contexte React afin que les écrans sachent si une action nécessite une session active.

Le nom affiché est stocké dans la table `profiles`. Cette table est liée à l'utilisateur Supabase Auth par son identifiant. Elle permet d'afficher un nom lisible dans l'interface, par exemple "Thomas", au lieu de montrer l'adresse email complète.

Un déclencheur SQL crée automatiquement une ligne de profil lorsqu'un nouvel utilisateur est ajouté dans `auth.users`. Le service d'authentification vérifie aussi que le profil existe après connexion, ce qui évite d'avoir une interface sans nom si une ligne manque.

Le choix d'un compte utilisateur est important pour Bidly : il permet de savoir qui vend un objet, qui place une offre, qui est le meilleur enchérisseur et qui doit recevoir une notification de surenchère.

5. Création et gestion des ventes

La création d'une vente se fait dans l'écran dédié. Le formulaire reste volontairement simple : titre, description, prix de départ, date de fin et image optionnelle. Ces champs suffisent pour présenter une enchère complète sans ajouter des notions hors périmètre comme la livraison ou le paiement.

L'image peut être fournie de deux manières. L'utilisateur peut coller une URL d'image, ou choisir une image depuis son appareil. L'application prépare ensuite cette valeur pour l'afficher avec le composant Image de React Native.

Lorsqu'une vente est créée, elle est associée à l'utilisateur connecté avec `seller_id` et `seller_name`. Cette information est utilisée pour afficher le vendeur, empêcher le vendeur d'enchérir sur sa propre vente et autoriser la suppression uniquement par le vendeur.

La suppression d'une vente est disponible sur l'écran de détail pour le vendeur. Elle repose aussi sur une règle Row Level Security côté Supabase, afin qu'un utilisateur ne puisse pas supprimer une vente qui ne lui appartient pas.

6. Placement d'une offre

Le placement d'une offre est la règle métier principale du projet. Le code vérifie les erreurs évidentes côté interface pour donner un retour rapide : montant vide, montant non numérique, montant trop faible, utilisateur non connecté, enchère terminée ou vente appartenant à l'utilisateur courant.

La validation finale est réalisée côté Supabase avec la fonction SQL `place_bid`. Cette fonction relit l'état actuel de l'enchère avant d'accepter l'offre. Cela évite de se baser uniquement sur une donnée affichée dans l'application, qui pourrait être dépassée si un autre utilisateur a surenchéri entre-temps.

La fonction vérifie notamment les règles suivantes.

- l'utilisateur doit être connecté ;
- l'enchère doit exister ;
- l'enchère doit être active ;
- la date de fin ne doit pas être dépassée ;
- l'utilisateur ne doit pas être le vendeur ;
- le montant doit être supérieur au prix actuel ;
- la première offre doit respecter le prix de départ.

Si toutes les vérifications réussissent, la fonction insère la nouvelle offre dans `bids`, met à jour le prix actuel dans `auction_items` et enregistre le meilleur enchérisseur.

7. Notifications et suivi

Les notifications sont internes à l'application. Elles sont stockées dans la table `notifications` avec l'utilisateur concerné, l'enchère, le message, l'état lu/non lu et la date de création.

Une notification est créée uniquement lorsqu'un utilisateur différent dépasse l'ancien meilleur enchérisseur. Si personne n'avait encore enchéri, aucune notification n'est nécessaire. Si un utilisateur augmente lui-même son offre, l'application ne crée pas de notification de surenchère pour lui.

L'écran Notifications affiche les messages reçus par l'utilisateur connecté. Le badge dans la navigation indique le nombre de notifications non lues, ce qui donne un retour visuel immédiat.

L'écran Mes offres complète ce système. Il regroupe les enchères sur lesquelles l'utilisateur a déjà placé une offre et indique si son offre est encore la meilleure. Cela permet de suivre facilement les enchères importantes sans devoir parcourir toute la liste.

8. Base de données Supabase

Le schéma SQL est défini dans `supabase/schema.sql`. Il crée les tables nécessaires au fonctionnement de Bidly : `profiles`, `auction_items`, `bids` et `notifications`.

La table `profiles` contient le nom public de l'utilisateur. La table `auction_items` contient les informations des ventes : titre, description, image, prix de départ, prix actuel, vendeur, meilleur enchérisseur, date de fin et statut.

La table `bids` contient l'historique des offres. Chaque ligne relie un utilisateur à une enchère avec un montant et une date. Cet historique permet d'afficher les dernières offres et de retrouver les enchères auxquelles un utilisateur a participé.

La table `notifications` contient les notifications de surenchère. Elle permet de garder les messages après rechargement de l'application et de distinguer les notifications lues des notifications non lues.

Des index sont ajoutés sur les identifiants souvent utilisés, par exemple les enchères liées aux offres, les vendeurs, les meilleurs enchérisseurs et les notifications d'un utilisateur. Cela garde les requêtes simples et plus efficaces.

9. Sécurité et Row Level Security

Supabase Row Level Security est activé sur les tables principales. Les règles restent compréhensibles pour un projet scolaire, mais elles protègent les actions importantes.

Les enchères actives peuvent être lues afin que la liste soit visible. Un utilisateur connecté peut créer une vente avec son propre identifiant comme vendeur. Un vendeur peut modifier ou supprimer uniquement ses propres ventes.

Les offres peuvent être lues par les utilisateurs connectés et insérées par l'utilisateur courant. Les notifications peuvent être lues et mises à jour uniquement par leur destinataire.

La clé utilisée côté application est la clé publique anon. Aucune clé service role n'est placée dans le code client. Les valeurs Supabase sont lues depuis les variables d'environnement `EXPO_PUBLIC_SUPABASE_URL` et `EXPO_PUBLIC_SUPABASE_ANON_KEY`.

10. Justification technique

React Native et Expo

React Native a été utilisé pour construire une application mobile avec une base de code unique. Expo simplifie le lancement du projet, les tests avec Expo Go, le mode web et l'accès à certaines fonctionnalités comme le choix d'image.

TypeScript

TypeScript permet de décrire clairement les structures utilisées dans l'application, par exemple les enchères, les offres et les notifications. Cela limite les erreurs et rend le code plus facile à expliquer.

Supabase plutôt qu'un stockage local

Supabase a été choisi car un système d'enchères nécessite des données partagées entre plusieurs utilisateurs. Deux comptes différents peuvent interagir avec la même enchère, voir le prix actuel et recevoir une notification après une surenchère.

Un stockage local serait plus simple pour un premier prototype, mais il ne permettrait pas de démontrer correctement une enchère entre plusieurs utilisateurs. Supabase fournit directement une base PostgreSQL, l'authentification et les règles de sécurité.

Notifications internes

Les notifications push natives n'ont pas été ajoutées. Elles demanderaient une configuration plus lourde et des identifiants de production. Pour ce projet, des notifications internes suffisent à démontrer la règle métier : prévenir un utilisateur lorsqu'il est dépassé.

11. Architecture du projet

Le projet garde une structure simple pour rester compréhensible pendant une défense orale. Les écrans ne contiennent pas directement toute la logique de base de données : les appels Supabase sont isolés dans des services.

- `app/` contient les écrans et la navigation Expo Router.
- `components/` contient les composants réutilisables comme les cartes d'enchères.
- `context/` contient les contextes React pour l'authentification et les enchères.
- `services/` contient les appels à Supabase et la logique d'accès aux données.
- `supabase/` contient le schéma SQL et les données de départ.
- `types/` contient les types TypeScript utilisés dans l'application.
- `utils/` contient les fonctions simples de formatage des prix, dates et statuts.

Le contexte d'authentification garde la session et le nom d'affichage. Le contexte des enchères charge les enchères, les offres et les notifications, puis expose les actions principales aux écrans.

Les services séparent les responsabilités : `auctionsService` charge, crée et supprime les enchères ; `bidsService` place les offres ; `notificationsService` gère les notifications ; `profilesService` récupère le nom d'affichage.

12. Flux de données importants

Flux de création d'une enchère

L'utilisateur remplit le formulaire de création. L'écran vérifie que les champs obligatoires sont présents et que le prix de départ est valide. Le service crée ensuite une ligne dans `auction_items` avec le vendeur connecté, le prix de départ, le prix actuel initial et la date de fin.

Flux de placement d'une offre

L'écran de détail vérifie les erreurs simples, puis appelle le contexte. Le contexte appelle `bidsService`, qui exécute `place_bid` côté Supabase. La base valide l'offre, insère la ligne dans `bids`, met à jour l'enchère et crée une notification si un ancien meilleur enchérisseur a été dépassé.

Flux de suivi

Les offres de l'utilisateur sont chargées depuis Supabase. L'écran Mes offres compare la dernière offre de l'utilisateur avec le meilleur enchérisseur actuel pour indiquer si l'utilisateur est en tête ou non.

Rafraîchissement des données

L'application recharge les données régulièrement afin que deux appareils connectés voient les nouveaux prix sans devoir relancer l'application. Ce choix reste plus simple qu'une architecture temps réel complète et suffit pour la démonstration.

13. Interface et expérience utilisateur

L'interface reste volontairement sobre. Les boutons d'inscription, de connexion et de déconnexion sont visibles et lisibles. L'utilisateur connecté est identifié par son nom, ce qui donne une interface plus claire qu'un affichage permanent de l'adresse email.

Les formulaires affichent des messages courts en cas d'erreur : champ manquant, montant invalide, offre trop basse, enchère terminée ou action interdite. L'objectif est que l'utilisateur comprenne immédiatement pourquoi une action ne passe pas.

Les écrans de données prévoient des états simples : chargement, erreur et absence de résultat. Cela évite les écrans vides et rend l'application plus facile à présenter.

Les prix sont affichés en euros et les dates sont formatées dans un style lisible. Ces petits utilitaires évitent de répéter le même formatage dans plusieurs écrans.

14. Auto-évaluation

Fonctionnalité annoncée	Réalisée ?	Commentaire
Liste des enchères	Oui	Les enchères actives sont affichées avec leur titre, image, prix actuel et statut.
Recherche par mot-clé	Oui	La liste est filtrée selon le texte saisi par l'utilisateur.
Détail d'une enchère	Oui	L'utilisateur peut consulter le titre, la description, les prix, la date de fin, le vendeur et les dernières offres.
Placement d'une offre	Oui	Un utilisateur connecté peut placer une offre sur une enchère active qui n'est pas sa propre vente.
Validation des enchères	Oui	Les montants invalides, trop bas, les enchères terminées et les offres sur sa propre vente sont refusés.
Notifications de surenchère	Oui	Une notification interne est créée pour l'ancien meilleur enchérisseur lorsqu'un autre utilisateur le dépasse.
Suivi des enchères	Oui	L'écran Mes offres affiche les enchères auxquelles l'utilisateur a participé et indique s'il est en tête.
Authentification	Oui	L'inscription et la connexion utilisent Supabase Auth avec email et mot de passe.
Création de vente	Oui	Un utilisateur connecté peut créer une enchère avec titre, description, prix, date de fin et image optionnelle.
Suppression de vente	Oui	Le vendeur peut supprimer uniquement ses propres ventes, avec une règle RLS côté Supabase.
Gestion de la fin des enchères	Oui	Les offres sont bloquées si la date de fin est dépassée ou si le statut n'est plus actif.
Temps réel avancé	Partiel	L'application recharge les données régulièrement. Un système temps réel complexe n'a pas été ajouté pour garder le projet simple.
Paieement et livraison	Non	Ces fonctionnalités sont hors périmètre pour ce projet.

15. Parcours de démonstration

1. Ouvrir l'application avec Expo Go ou la version web.
2. Consulter la liste des enchères.
3. Rechercher un objet par mot-clé.
4. Créer un compte ou se connecter.
5. Créer une nouvelle enchère avec une image.
6. Ouvrir sa propre vente et vérifier que le bouton d'offre est bloqué.
7. Se connecter avec un autre compte et placer une offre valide.
8. Essayer de placer une offre trop basse.
9. Surenchérir avec un autre utilisateur.
10. Vérifier la notification de surenchère.
11. Ouvrir Mes offres pour vérifier le suivi.
12. Supprimer une vente créée avec le compte vendeur.

16. Configuration et lancement

Le projet se lance avec Expo. Le fichier `.nvmrc` indique la version de Node utilisée pour stabiliser l'environnement de développement. Le lancement recommandé consiste à se placer dans le dossier du projet, utiliser la version Node indiquée, installer les dépendances puis lancer Expo.

Les commandes principales sont les suivantes.

- `nvm use` pour utiliser la version de Node du projet ;
- `npm install` pour installer les dépendances ;
- `npm start` pour lancer Expo Go et scanner le QR code ;
- `npm run web` pour ouvrir l'application dans le navigateur.

Les identifiants Supabase ne sont pas écrits directement dans le code. Ils doivent être placés dans un fichier `.env` local avec les clés `EXP0_PUBLIC_SUPABASE_URL` et `EXP0_PUBLIC_SUPABASE_ANON_KEY`. Ce fichier ne doit pas être envoyé sur GitHub.

Pour préparer une base Supabase propre, le fichier `supabase/schema.sql` doit être exécuté dans l'éditeur SQL de Supabase. Le fichier `supabase/seed.sql` peut ensuite être exécuté pour ajouter quelques enchères de démonstration.

17. Vérifications manuelles

Les vérifications suivantes permettent de confirmer que le projet répond aux fonctionnalités annoncées.

Parcours testé	Résultat attendu
Ouvrir l'application	La liste des enchères se charge depuis Supabase.
Rechercher un mot-clé	La liste se filtre selon le texte saisi.
Créer un compte	Le profil est créé et le nom s'affiche dans l'application.
Créer une vente	La vente apparaît dans la liste avec son prix et son image.
Enchérir sur sa propre vente	L'action est bloquée.
Placer une offre trop basse	L'offre est refusée avec un message d'erreur clair.
Placer une offre valide	Le prix actuel augmente et l'offre apparaît dans l'historique.
Surenchérir avec un autre compte	L'ancien meilleur enchérisseur reçoit une notification interne.
Ouvrir Mes offres	L'utilisateur voit les enchères auxquelles il a participé.
Supprimer sa vente	La vente disparaît de la liste et ne peut pas être supprimée par un autre utilisateur.

18. Données de démonstration

Le fichier `supabase/seed.sql` contient des enchères de départ pour faciliter la démonstration. Ces données évitent de présenter une application vide lors du premier lancement.

Les objets de démonstration restent volontairement simples et réalistes : casque audio, montre connectée, sac à dos, clavier mécanique, jeu vidéo ou appareil photo compact. Chaque enchère contient un titre, une description, un prix de départ, un prix actuel, une image et une date de fin.

Ces données ne remplacent pas les vraies actions utilisateur. Une vente créée depuis l'application est enregistrée dans la même table Supabase que les données de départ, ce qui permet de montrer que l'application utilise bien une source distante.

19. Limites connues

Le projet ne gère pas les paiements, la livraison, les litiges ou les évaluations. Ces éléments sont volontairement exclus, car Bidly est un prototype d'enchères simples et non une marketplace complète.

Les notifications sont internes à l'application. Elles ne sont pas envoyées sous forme de notifications push natives lorsque l'application est fermée. Ce choix évite une configuration de production plus lourde.

Le rafraîchissement des données reste simple. L'application recharge les informations à intervalle régulier plutôt que d'utiliser une architecture temps réel avancée. Cette solution est suffisante pour la démonstration et plus facile à expliquer.

Les images choisies depuis l'appareil sont adaptées à un prototype. Une version plus avancée pourrait utiliser Supabase Storage, mais cela ajouterait une couche de configuration supplémentaire.

20. Améliorations possibles

Une première amélioration serait d'ajouter des tests automatisés sur la logique d'offres : montant trop bas, enchère terminée, vendeur qui tente d'enchérir, notification de l'ancien meilleur enchérisseur.

Une deuxième amélioration serait d'utiliser Supabase Realtime de manière plus complète pour mettre à jour instantanément les prix, les offres et les notifications, sans rechargement périodique.

Une troisième amélioration serait d'ajouter Supabase Storage pour stocker proprement les images de vente au lieu de dépendre d'URL ou de données intégrées.

Une quatrième amélioration serait d'améliorer la gestion des enchères terminées avec un affichage plus détaillé du gagnant et un historique plus complet.

Enfin, l'interface pourrait être encore améliorée avec une meilleure hiérarchie visuelle, des confirmations plus détaillées et une navigation plus confortable sur les petits écrans.

21. Conclusion

Bidly couvre les fonctionnalités principales attendues pour une application mobile d'enchères simples : consulter des ventes, rechercher un objet, créer une vente, placer une offre, valider les règles métier, suivre ses offres et recevoir une notification lorsqu'un autre utilisateur surenchérit.

Le choix de Supabase est cohérent avec le besoin multi-utilisateur du projet. Il permet de partager les données entre plusieurs appareils, de gérer l'authentification et de sécuriser les actions importantes avec Row Level Security.

L'architecture reste volontairement simple : écrans Expo Router, composants réutilisables, contextes React, services Supabase, types TypeScript et utilitaires de formatage. Cette organisation permet de comprendre le projet et de le défendre clairement.